

Les enregistrements et les listes chaînées

B.U.T. Informatique

Objectifs d'apprentissage

A la fin de ce chapitre (CM + TDs + TPs), vous serez capable de :

- savoir quand et comment rassembler plusieurs variables de types différent sous un même nom (enregistrement)
- utiliser des enregistrements en fonction du problème traité
- définir et créer une liste chaînée de valeurs
- manipuler une liste chaînée

Enregistrement

Un enregistrement

Agrégation de plusieurs variables dans une seule. C'est un type de variable personnalisé qui peut contenir des valeurs de plusieurs types

- Permet d'associer plusieurs valeurs à un seul nom de variable
- Différent d'un tableau car des types différents
- Les "sous-variables" sont appelées des champs
- Utilisation de la notion pointée pour accéder aux champs
- Première notion d'objet (semestre 2)

Exemple

```
struct coord {
    int x;
    int y;
};

int main() {

    coord pointA;
    pointA.x = 15;
    pointA.y = 21;

    std::cout<< " Les_cooronnees_de_A_sont_:";
    std::cout<< "("<<pointA.x<< " , "<<pointA.y<< ")"<<std::endl;
}
```

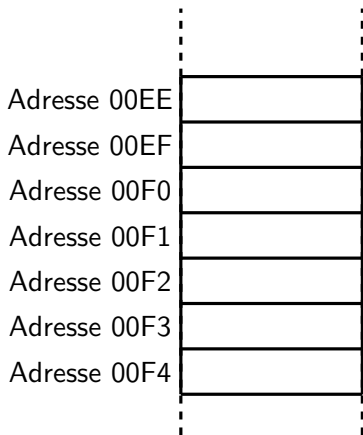
Exemple

```
1  #include<iostream>
2
3
4  struct weapon {
5      int damage;
6      std::string name;
7  };
8
9
10 struct statChar {
11     int HP;
12     std::string name;
13     weapon* stuff;
14 };
15
16
17 int main() {
18
19     weapon sword;
20     sword.damage = 2;
21     sword.name = "epee magique tueuse de goblin (inhabituel)";
22
23     statChar george;
24
25     george.name = "George, Le Pourfendeur De Mamans";
26     george.HP = 50;
27     george.stuff = &sword;
28
29     std::cout << george.name << " possede " << george.HP << " points de vie" << std::endl;
30     std::cout << george.name << " possede " << george.stuff->name << " comme équipement" << std::endl;
31
32
33
```

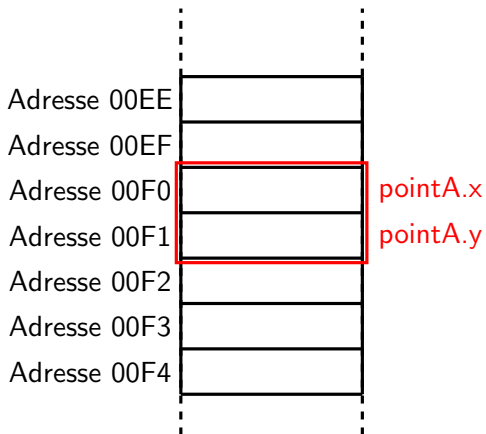
Enregistrement

- '.' : permet d'accéder à la valeur d'un champs de notre variable
- '->' : permet d'accéder à la valeur d'un champs de la variable pointé par notre pointeur (similaire à *)

Exemple

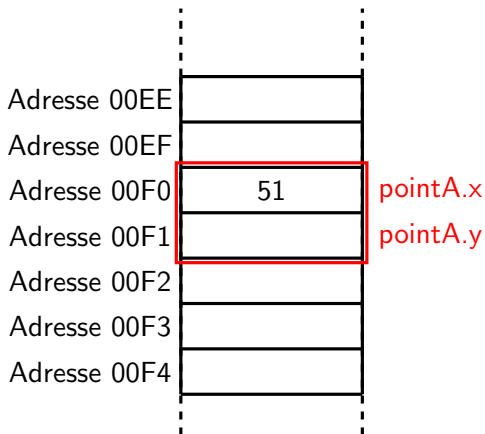


Exemple



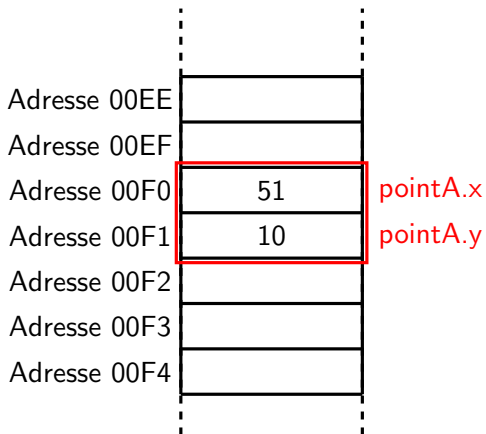
1 coord pointA;

Exemple



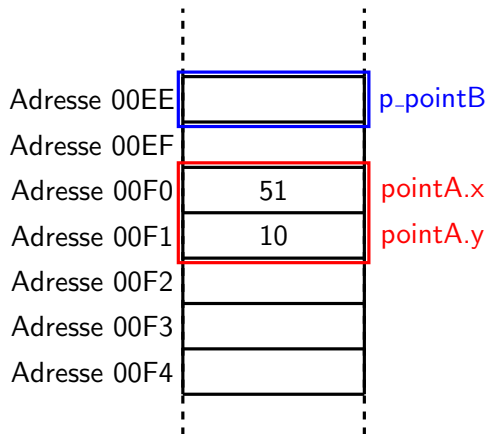
- 1 `coord pointA;`
- 2 `pointA.x = 51;`

Exemple



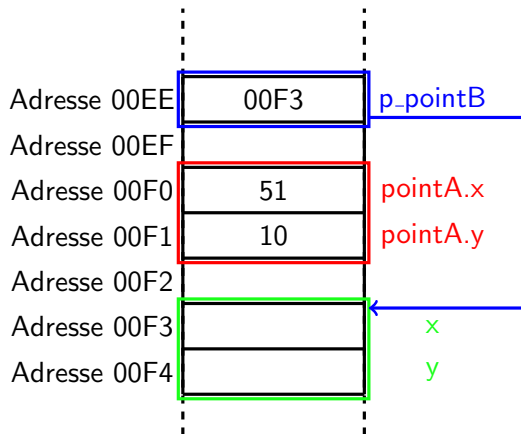
- 1 coord pointA;
- 2 pointA.x = 51;
- 3 pointA.y = 10;

Exemple



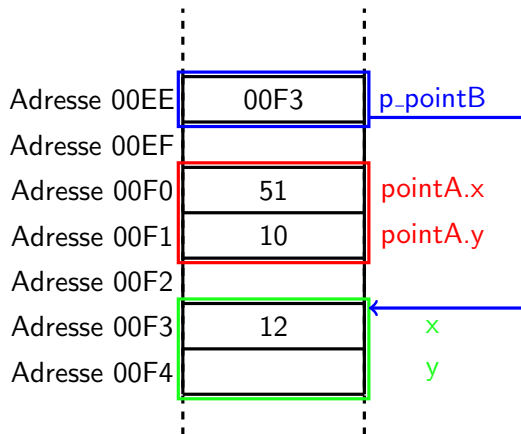
- 1 coord pointA;
- 2 pointA.x = 51;
- 3 pointA.y = 10;
- 4 coord *p_pointB;

Exemple



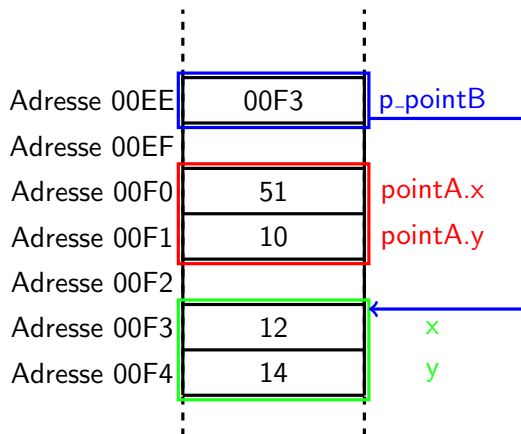
- 1 coord pointA;
- 2 pointA.x = 51;
- 3 pointA.y = 10;
- 4 coord *p_pointB;
- 5 p_pointB = new coord;

Exemple



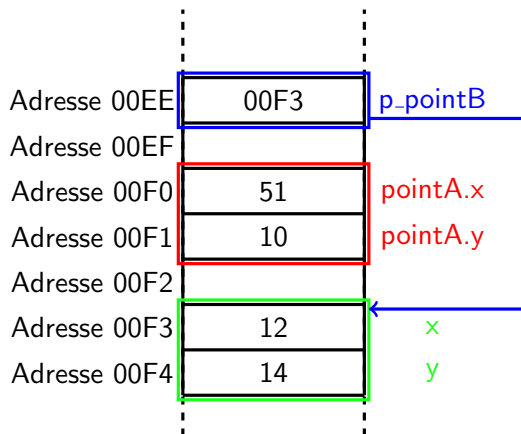
- 1 coord pointA;
- 2 pointA.x = 51;
- 3 pointA.y = 10;
- 4 coord *p_pointB;
- 5 p_pointB = new coord;
- 6 p_pointB->x = 12;

Exemple



- 1 coord pointA;
- 2 pointA.x = 51;
- 3 pointA.y = 10;
- 4 coord *p_pointB;
- 5 p_pointB = new coord;
- 6 p_pointB->x = 12;
- 7 p_pointB->y = 14;

Exemple



- 1 coord pointA;
- 2 pointA.x = 51;
- 3 pointA.y = 10;
- 4 coord *p_pointB;
- 5 p_pointB = new coord;
- 6 p_pointB->x = 12;
- 7 p_pointB->y = 14;

p_pointB ? p_pointB.x ?
p_pointB->y ? &pointA ?

Les listes chaînées

- Un tableau nécessite des cases continues dans la mémoire
- Ce n'est pas facile d'ajouter des éléments
- Et si on utilisait les pointeurs ?

Les listes chaînées

- Un tableau nécessite des cases continues dans la mémoire
- Ce n'est pas facile d'ajouter des éléments
- Et si on utilisait les pointeurs ?
- Les listes chaînées !

Les listes chaînées

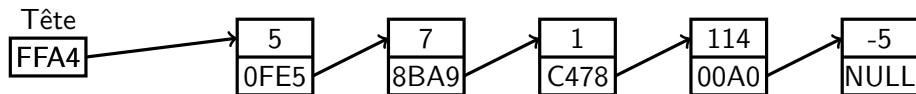
Une liste chaînée

Une liste chaînée est une suite de maillons, tel que chaque maillon stocke une valeur et pointe vers l'espace mémoire du maillon suivant

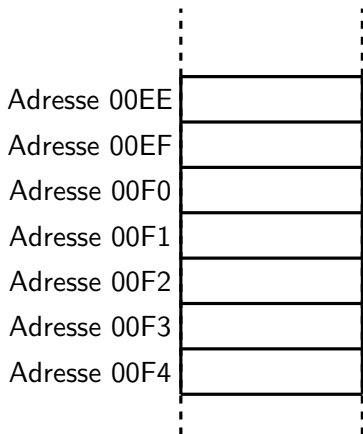
- Un maillon est un enregistrement
- Liste (simple) chaînée = un pointeur vers maillon suivant
- Liste (doublement) chaînée = un pointeur vers maillon suivant et un vers maillon précédent
- Un cycle = liste chaînée dont le dernier élément pointe sur le premier
- La collection de valeurs est seulement retenue grâce à un pointeur qui pointe vers le premier maillon de la chaîne (la tête)

Exemple

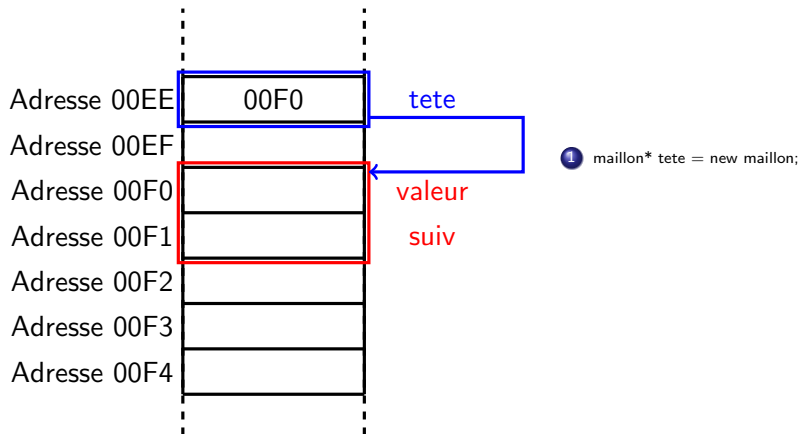
```
struct maillon {  
    int valeur;  
    maillon* suiv;  
};
```



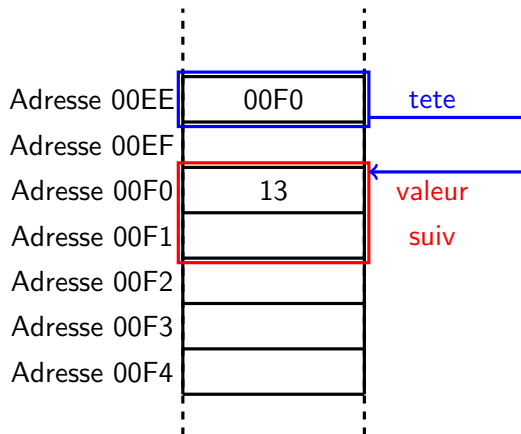
Exemple



Exemple

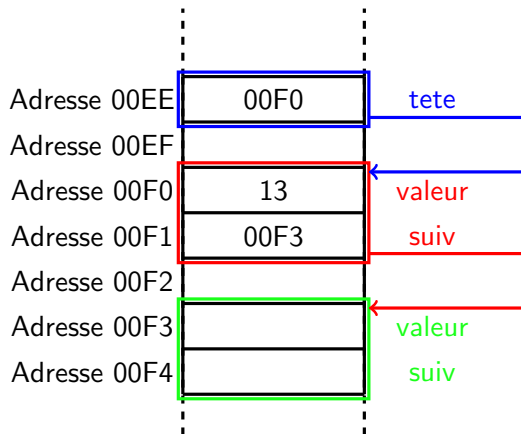


Exemple



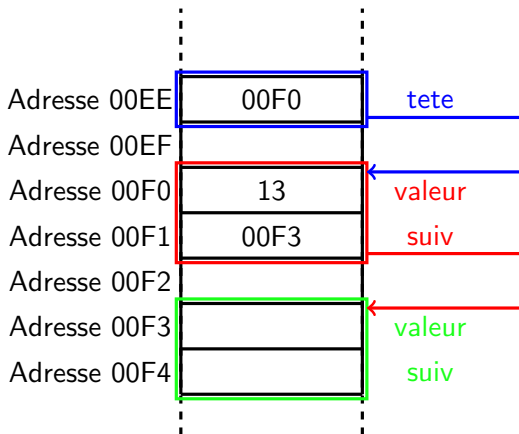
- 1 `maillon* tete = new maillon;`
- 2 `tete->valeur = 13;`

Exemple



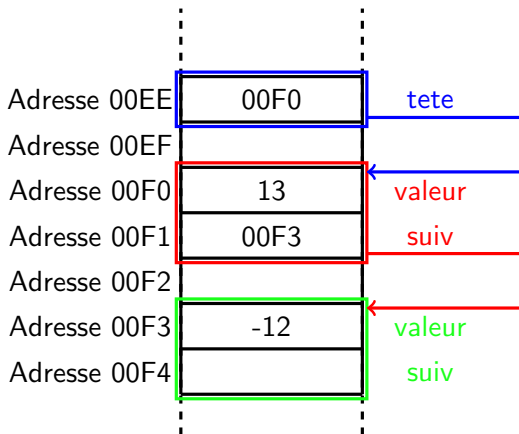
- 1 `maillon* tete = new maillon;`
- 2 `tete->valeur = 13;`
- 3 `tete->suiv = new maillon;`

Exemple



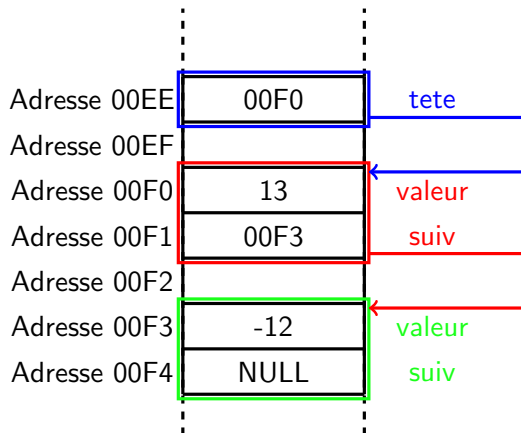
- 1 maillon* tete = new maillon;
- 2 tete->valeur = 13;
- 3 tete->suiv = new maillon;
- 4 maillon *courant = tete->suiv;

Exemple



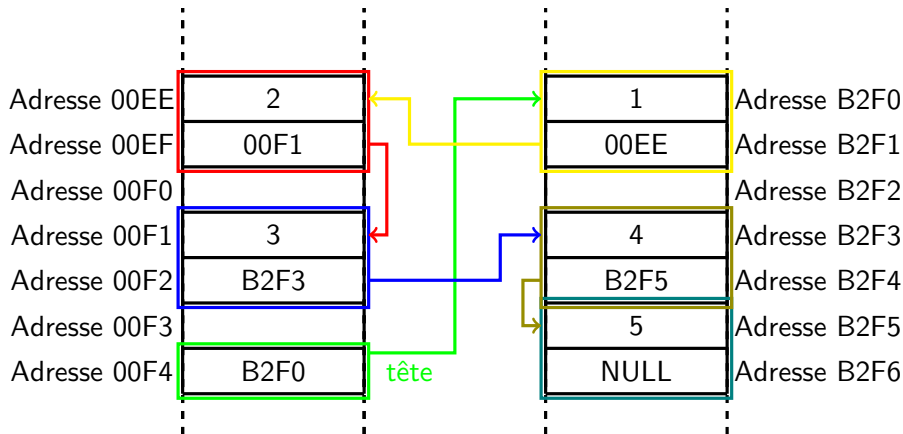
- 1 `maillon* tete = new maillon;`
- 2 `tete->valeur = 13;`
- 3 `tete->suiv = new maillon;`
- 4 `maillon *courant = tete->suiv;`
- 5 `courant->valeur = -12;`

Exemple



- 1 maillon* tete = new maillon;
- 2 tete->valeur = 13;
- 3 tete->suiv = new maillon;
- 4 maillon *courant = tete->suiv;
- 5 courant->valeur = -12;
- 6 courant->suiv = NULL;

Exemple



Exemple

```
maillon* tete = new maillon;

tete->valeur = 15;
tete->suiv = new maillon;

tete->suiv->valeur = 255;
tete->suiv->suiv = NULL;

maillon *courant = tete->suiv;

Afficher(tete); Afficher(tete->suiv); // ?
Afficher(tete->valeur); Afficher(tete->suiv->valeur); // ?
Afficher(tete->suiv->suiv); Afficher(courant); // ?
Afficher(courant->suiv); Afficher(courant->valeur); // ?

delete tete->suiv;
delete tete;
```

Pour utiliser des listes chaînées, il est nécessaire de produire des primitives :

- ❶ Placer une valeur en tant que premier élément de la liste (Ajout tête)
- ❷ Placer une valeur en tant que dernier élément de la liste (Ajout queue)
- ❸ Placer une valeur après ou avant un élément donné de la liste (Insertion)
- ❹ Vérifier si la liste est vide
- ❺ Compter le nombre d'éléments
- ❻ Supprimer un élément

Discutons complexité

Opération	Tableau dynamique	Liste chaînée (avec tête et queue)
Accès à une donnée Recherche d'une donnée Insertion donnée Ajout d'une donnée début/fin Suppression d'une donnée Suppression d'une donnée début Suppression d'une donnée fin		

Discutons complexité

Opération	Tableau dynamique	Liste chaînée (avec tête et queue)
Accès à une donnée Recherche d'une donnée Insertion donnée Ajout d'une donnée début/fin Suppression d'une donnée Suppression d'une donnée début Suppression d'une donnée fin	$O(1)$	$O(n)$

Discutons complexité



Opération	Tableau dynamique	Liste chaînée (avec tête et queue)
Accès à une donnée	$O(1)$	$O(n)$
Recherche d'une donnée	$O(n)$	$O(n)$
Insertion donnée		
Ajout d'une donnée début/fin		
Suppression d'une donnée		
Suppression d'une donnée début		
Suppression d'une donnée fin		

Discutons complexité

Opération	Tableau dynamique	Liste chaînée (avec tête et queue)
Accès à une donnée	$O(1)$	$O(n)$
Recherche d'une donnée	$O(n)$	$O(n)$
Insertion donnée	$O(n)$	$O(n)$ ou $O(1)$
Ajout d'une donnée début/fin		
Suppression d'une donnée		
Suppression d'une donnée début		
Suppression d'une donnée fin		

Discutons complexité

Opération	Tableau dynamique	Liste chaînée (avec tête et queue)
Accès à une donnée	$O(1)$	$O(n)$
Recherche d'une donnée	$O(n)$	$O(n)$
Insertion donnée	$O(n)$	$O(n)$ ou $O(1)$
Ajout d'une donnée début/fin	$O(n)$	$O(1)$
Suppression d'une donnée		
Suppression d'une donnée début		
Suppression d'une donnée fin		

Discutons complexité

Opération	Tableau dynamique	Liste chaînée (avec tête et queue)
Accès à une donnée	$O(1)$	$O(n)$
Recherche d'une donnée	$O(n)$	$O(n)$
Insertion donnée	$O(n)$	$O(n)$ ou $O(1)$
Ajout d'une donnée début/fin	$O(n)$	$O(1)$
Suppression d'une donnée	$O(n)$	$O(n)$
Suppression d'une donnée début		
Suppression d'une donnée fin		

Discutons complexité

Opération	Tableau dynamique	Liste chaînée (avec tête et queue)
Accès à une donnée	$O(1)$	$O(n)$
Recherche d'une donnée	$O(n)$	$O(n)$
Insertion donnée	$O(n)$	$O(n)$ ou $O(1)$
Ajout d'une donnée début/fin	$O(n)$	$O(1)$
Suppression d'une donnée	$O(n)$	$O(n)$
Suppression d'une donnée début	$O(n)$	$O(1)$
Suppression d'une donnée fin	$O(1)$	$O(1)$

Comparaison

Avantages :

- Tableau
 - Accès rapide à une valeur
 - Peu gourmand en mémoire
 - Efficace avec les caches (localité spatiale)
- Liste Chaînée
 - Taille dynamique
 - Utilise uniquement la mémoire nécessaire
 - Insertion et suppression rapide (tête et queue)

Défauts :

- Tableau
 - Taille fixe
 - Insertion et suppression
 - Alloue plus de mémoire que nécessaire
- Liste Chaînée
 - Accès à une valeur
 - Gourmand en mémoire
 - Peu efficace niveau matériel